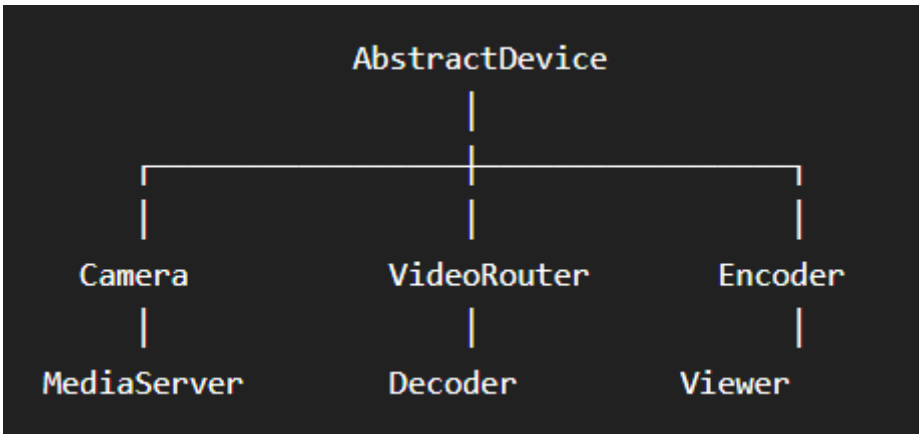
Step 3 — Device Framework (Core Design)

Goal

Every device (Camera, Router, Encoder, etc.) should behave consistently.

Instead of each device having completely different code, they all inherit from a common base class.



---

1. Device Lifecycle

Every device follows the same lifecycle.

---

2. Common Properties

Every device has these fields:

deviceId

name

deviceType

status

xPosition

yPosition

powerConsumption

temperature

cpuUsage

memoryUsage

health

lastUpdated

Notice that xPosition and yPosition allow the frontend to draw the device on the topology canvas.

---

3. Common Methods

Every device must implement these operations.

initialize()

update()

applyEvent()

recalculate()

receiveSignal()

sendSignal()

Meaning:

- initialize() → Set default values.
- update() → Called every simulation tick.
- applyEvent() → Handle events like Disconnect or Restart.
- recalculate() → Update dependent properties.
- receiveSignal() → Accept input from connected devices.
- sendSignal() → Produce output for downstream devices.

---

4. Input & Output Ports

Instead of directly connecting devices, connect their ports.

Example:

Camera(Output)

|

Router(Input)

Router(Output)

|

Encoder(Input)

This makes the simulator extensible.

Later you can support devices with multiple inputs/outputs without redesigning everything.

---

5. Device State

Each device keeps:

ONLINE

WARNING

FAILED

RECOVERING

The Alarm Engine will use these states.

---

6. Device Profile

Each device also has a profile.

Example:

Generic Camera

Resolution : 1080p

FPS : 30

Power : 5W

Another profile:

4K Camera

Resolution : 4k

FPS : 60

Power : 18W

Creating a new device means selecting a profile instead of manually configuring every property.

---

7. Device Events

Every device supports common events:

Connect

Disconnect

Restart

Recover

Device-specific events:

Camera

- Signal Loss

Encoder

- High CPU
- Overheating

Router

- Route Failure

This keeps common behavior in one place while allowing specialization.

---

8. Device Metrics

Every device periodically reports:

CPU

Memory

Temperature

Power

Status

The frontend simply displays these metrics.

---

9. Device Connections

Instead of storing references directly:

Camera → Router

We'll have a Connection object:

Connection

sourceDevice

sourcePort

targetDevice

targetPort

status

This is cleaner and makes deleting or validating connections easier.

---

10. Design Principles

The framework should follow:

- Single Responsibility Principle: each class has one clear purpose.
- Open/Closed Principle: new device types can be added without modifying the simulation engine.
- Composition where appropriate: connections, profiles, and rules are separate objects rather than being embedded everywhere.

Every device contains:

- Identity
- Properties
- Metrics
- Ports
- State

- Profile
- Events

and exposes:

- initialize()
- update()
- applyEvent()
- recalculate()
- receiveSignal()
- sendSignal()

Before Moving On: One Important Improvement

I want to introduce Device Properties as reusable objects instead of hardcoding fields.

Instead of:

camera.setFps(60);

camera.setTemperature(45);

camera.setPower(10);

we'll model properties like this conceptually:

Property

name

value

unit

editable

minimum

maximum

Examples:

- FPS

- Resolution
- Bitrate
- Temperature
- CPU Usage

This has a big advantage: the frontend can automatically build the property panel from the property list, and adding a new property later won't require changing the UI or core engine.